

项目：基于用户历史数据与行为特征的信贷违约预测模型

作者：王子安（阿里线上数据分析实习）

周期：2026/8/31 – 2026/9/27（W1-W4）

数据：阿里天池信贷违约数据集 [1]（train 80 万 / testA 20 万）

目录

|                                  |    |
|----------------------------------|----|
| 摘要 .....                         | 2  |
| 1. 项目概述与业务问题 .....               | 2  |
| 1.1 业务背景 .....                   | 2  |
| 1.2 要解决什么问题 .....                | 2  |
| 1.3 建模任务定义 .....                 | 2  |
| 2. 数据说明 .....                    | 2  |
| 2.1 数据来源与规模 .....                | 2  |
| 2.2 字段分类 .....                   | 2  |
| 2.3 目标变量分布 .....                 | 3  |
| 3. 数据预处理与质量控制 .....              | 3  |
| 3.1 处理决策表 .....                  | 3  |
| 3.2 异常值三层验证 .....                | 3  |
| 3.3 清洗结果 .....                   | 3  |
| 4. 探索性数据分析 .....                 | 3  |
| 4.1 信用等级是违约率最强的分层变量 .....        | 3  |
| 4.2 违约组 vs 正常组：均值画像 .....        | 4  |
| 4.3 收入严重右偏 → 特征工程做 log 变换 .....  | 4  |
| 4.4 关系偏非线性 .....                 | 4  |
| 5. 特征工程 .....                    | 4  |
| 5.1 类别编码 .....                   | 5  |
| 5.2 数值变换 .....                   | 5  |
| 5.3 特征创建 .....                   | 5  |
| 5.4 特征选择 .....                   | 5  |
| 5.5 防数据泄漏 .....                  | 5  |
| 5.6 方法取舍 .....                   | 5  |
| 5.7 特征数量实验 .....                 | 6  |
| 6. 基线模型构建与对比 .....               | 6  |
| 6.1 数据划分 .....                   | 6  |
| 6.2 三个基线模型 .....                 | 7  |
| 6.3 评估指标怎么读 .....                | 7  |
| 6.4 验证集三模型对比 .....               | 7  |
| 7. 模型优化 .....                    | 7  |
| 7.1 调参策略：为什么用随机搜索 .....          | 7  |
| 7.2 交叉验证 .....                   | 7  |
| 7.3 模型集成 .....                   | 7  |
| 7.4 优化前后对比 .....                 | 7  |
| 7.5 最终模型选择 .....                 | 8  |
| 8. 模型解释：SHAP 特征归因 .....          | 8  |
| 9. 最终模型评估 .....                  | 8  |
| 9.1 测试集成绩单 .....                 | 8  |
| 9.2 混淆矩阵 .....                   | 9  |
| 9.3 ROC 与 PR 曲线 .....            | 9  |
| 9.4 泛化能力分析 .....                 | 9  |
| 9.5 概率校准：让"概率"能直接算钱 .....        | 9  |
| 9.6 时间外验证 .....                  | 10 |
| 10. 风险阈值与定价空间 .....              | 10 |
| 10.1 固定阈值对照表 .....               | 10 |
| 10.2 从"拍一条线"到"按笔算账"：期望损失框架 ..... | 11 |
| 10.3 落在线上的人交人工：8,697 人的复审带 ..... | 13 |
| 10.4 延伸测算：低风险客户的让价空间 .....       | 13 |
| 11. 业务建议汇总 .....                 | 14 |
| 12. 局限性与后续工作 .....               | 14 |
| 12.1 现有局限的解决路径 .....             | 14 |
| 12.2 增强方向 .....                  | 15 |
| 12.3 落地顺序 .....                  | 16 |
| 13. 交付物与复现方式 .....               | 16 |
| 14. AI 工具使用声明 .....              | 16 |
| 15. 参考文献 .....                   | 17 |
| 附录：关键数字速查 .....                  | 18 |

## 摘要

模型给每个借款人打一个 0-1 的违约概率，机构用它把高风险客户排在名单前面，配合人工审核做风控。

最终模型（LightGBM 优化版）在从未参与建模的 79,850 人测试集上  $AUC = 0.7249$ 、 $KS = 0.3269$ ：随机抽一对“违约 vs 正常”客户，模型有 72.5% 的概率把违约者排在前面。

名单头部比整体更有用：分数最高的 10% 抓到全部违约者的 24.1%，是随机挑的 2.41 倍；前 30% 抓到 54.6%。

按放款日期做时间外验证（2016 年底前训练、2017-01 起打分）： $AUC$  从 0.7279 降到 0.7184，排序能力没垮；这段时间违约率 22.3%，比随机切分高 2.3 个百分点，概率轴要定期重标定。

审批建议给出两套放贷方案：方案① 单线扫描——只能设一条线就用 0.55，期望利润 6,340 万元；方案② 逐笔期望定线——能拆线就按期限拆成两条（3 年期 0.50、5 年期 0.62），期望利润 6,611 万元（模拟参数口径），比最优单线多 267 万元。方案② 落地时，被判拒的人里贴着线的一段（8,697 人）交人工复审，两头由机器判。

模型最看重的信息是贷款条件本身：子等级、期限、等级、利率、FICO 信用分排在前列，与业务直觉和探索阶段结论互相印证。

## 1. 项目概述与业务问题

### 1.1 业务背景

信贷审批的传统做法靠人工经验和规则清单，速度慢，借款人之间的差异也难量化（信贷评分的统计方法综述见 [12]，同类信贷数据上的算法横向比较见 [13]）。本项目目标是：利用借款人的历史数据与行为特征，预测违约概率，为金融机构的风控决策提供数据支持。

### 1.2 要解决什么问题

把“事后亏钱”变成“事前识别”：机构在放款前，用模型判断一个借款人有多大概率还不上钱，据此做两件事：

拒绝高风险客户（10.1 至 10.3 节）；

把有限的审核人力集中在“模型拿不准”的中间人群（10.3 节）。

给谁什么价、给多少额度不在本项目范围内；10.4 节的让价空间测算只说明这根概率轴还能怎么用，不构成定价方案。

### 1.3 建模任务定义

预测目标：`isDefault`（0 = 正常还款，1 = 违约）。

问题类型：二分类监督学习（预测“是否违约”），并输出违约概率。

评估立场：由于违约客户只占 20%（类别不平衡），全程以精确率 / 召回率 / F1 / AUC 为主要指标，准确率仅作参考。

## 2. 数据说明

### 2.1 数据来源与规模

| 数据集       | 原始               | 清洗后              | 说明                        |
|-----------|------------------|------------------|---------------------------|
| train.csv | 800,000 行 × 47 列 | 798,498 行 × 47 列 | 含目标变量 `isDefault`，用于训练与验证 |
| testA.csv | 200,000 行 × 46 列 | 199,623 行 × 46 列 | 无 `isDefault`，仅用于结构校验     |

重要说明：testA 是比赛用的“线上预测集”，没有违约标签，无法直接评估模型。因此本项目按任务文档要求，从 train 中切出 10%（79,850 人）作为测试集，从头到尾不参与任何建模决策，用于最终评估；testA 仅用于验证训练集分布可外推。

### 2.2 字段分类（47 列）

| 类别    | 字段                                        | 业务含义          |
|-------|-------------------------------------------|---------------|
| 身份/合同 | id、loanAmnt、term、interestRate、installment | 贷款金额、期限、利率、月供 |

|       |                                                    |                          |
|-------|----------------------------------------------------|--------------------------|
| 信用资质  | grade、subGrade、fico、dti、annualIncome、homeOwnership | 等级、信用分、负债收入比、收入、房产       |
| 历史行为  | delinquency_2years、openAcc、revolUtil、totalAcc 等    | 逾期史、账户数、额度使用率            |
| 地区/申请 | regionCode、purpose、applicationType、issueDate 等     | 地区、用途、申请类型、时间            |
| 匿名特征  | n0-n14 (15 个)                                      | 行为计数特征的脱敏处理，含义未公布，只能黑箱使用 |

### 2.3 目标变量分布（类别不平衡，重点）

正常还款：639,211 人（80.0%）

违约：159,287 人（20.0%）

每 5 个借款人里约 1 个违约。这种不平衡意味着：全猜"正常"也能有 80% 准确率，却一个违约客户都抓不到，所以评估必须看精确率/召回率/F1/AUC（详见第 6 节）。

## 3. 数据预处理与质量控制

遵循"先检测、后处理、能补不删"原则：先跑异常值检测报告，再依据结论执行清洗。完整日志见`outputs/W1/outlier\_report.txt`。

### 3.1 处理决策表

| 问题类型               | 发现 (train)       | 处理方式    | 理由              |
|--------------------|------------------|---------|-----------------|
| 年收入为 0             | 229 个            | 视为缺失并填充 | 逻辑上不可能，属记录错误    |
| 循环利用率 >100%        | 2,719 个          | 封顶到 100 | 保留样本信息，只修正越界    |
| dti 越界 (<0 或 >100) | 325 行            | 删除      | 脏数据             |
| 缺失极少列 (title 等)    | 1,177 行          | 删除      | 占比 <0.15%，删行无影响 |
| 匿名特征 n0-n14 缺失     | 约 40 万格 (约 5%/列) | 中位数填充   | 防止大量删行          |
| 就业年限缺失             | 46,375 个         | 填"未知"类别 | 保留为有效取值         |
| id 重复 / fico 上下限倒挂 | 0                | —       | 数据本身一致          |

### 3.2 异常值三层验证（为什么"抓到了却不删"）

箱线图/IQR 与  $3\sigma$  检测抓到的极端值（高收入、高余额等），经三层检验确认为真实业务极端客户：① 业务规则已先排除不可能的值；② 极端样本画像跨列自洽（收入最高者的 FICO、dti 均正常）；③ 极端值呈连续长尾而非孤立跳变。因此保留不删，后续用 log 变换压缩其影响。

### 3.3 清洗结果

行数：800,000 → 798,498（仅删 0.19%，样本量完整保留——清洗修的是"格子质量"，不是砍样本）。

列数：47 列（清洗阶段不变，特征选择在 W2 完成）。

成果：无缺失、无矛盾、无越界，数据可直接进入特征工程。

## 4. 探索性数据分析（EDA）

图见`outputs/W1/eda\_\*.png`，关键发现四条，全部被后续建模印证。

### 4.1 信用等级是违约率最强的分层变量

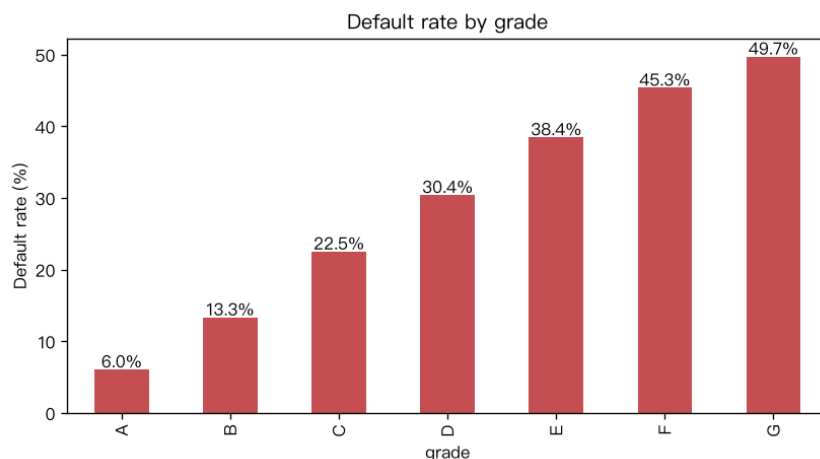


图 1 | 各信用等级的违约率：等级越高违约率单调上升

| grade | 样本数     | 违约率   |
|-------|---------|-------|
| A     | 139,462 | 6.0%  |
| B     | 233,330 | 13.3% |
| C     | 226,664 | 22.5% |
| D     | 119,162 | 30.4% |
| E     | 55,527  | 38.4% |
| F     | 19,004  | 45.3% |
| G     | 5,349   | 49.7% |

A→G 违约率从 6% 单调升到近 50%——与计量里“分组均值随分组变量单调变化”是同一逻辑。等级本身已浓缩大量风险信息，是后续模型的“头号种子特征”。

#### 4.2 违约组 vs 正常组：均值画像

| 特征               | 正常组    | 违约组    | 解读                 |
|------------------|--------|--------|--------------------|
| 利率 interestRate  | 12.6%  | 15.7%  | 违约客户通常拿到更高利率（风险溢价） |
| 年收入 annualIncome | 77,641 | 70,277 | 违约组收入更低            |
| 债务收入比 dti        | 17.7   | 20.1   | 违约组负债压力更大          |
| 循环利用率 revolUtil  | 51.1%  | 54.7%  | 违约组更“刷爆”额度         |
| FICO 下限          | 698    | 688    | 违约组信用分更低           |

#### 4.3 收入严重右偏 → 特征工程做 log 变换

`annualIncome` 均值 7.6 万 > 中位数 6.5 万，90% 人群收入 ≤12.5 万，但最大达 1,099 万。取 log 后分布接近钟形（图见 `outputs/W1/eda\_income\_log\_compare.png`），因此 W2 对金额类变量统一 log 变换，避免超高收入者牵着模型走。

#### 4.4 关系偏非线性

与违约的相关系数绝对值普遍不高（利率 +0.26 最强）——说明光靠线性相关解释不了全部，这是后续引入树模型（随机森林/XGBoost）的动机之一。

### 5. 特征工程

目标：把原始字段整理成模型“最吃得动”的输入，四步走：编码 → 变换 → 创建 → 选择。完整细节见 `reports/特征工程报告.md`。

### 5.1 类别编码

有序类别用序数编码：`grade`（1-7）、`subGrade`（1-35）、`employmentLength`（年限，未知=-1）。

无序类别用独热编码（`drop_first=True`，避免虚拟变量陷阱）：`homeOwnership`、`verificationStatus`、`purpose`。

高基数地区用目标编码（= 训练集该地区违约率），只从训练集学习，防泄漏。

### 5.2 数值变换

右偏金额列（贷款金额、月供、年收入、余额）做 `log1p` 变换。

连续数值列 `z-score` 标准化（均值/标准差只从训练集拟合）。

### 5.3 特征创建（每个新特征都有一句业务理由）

| 新特征                                                                          | 业务理由              |
|------------------------------------------------------------------------------|-------------------|
| <code>`fico_avg`</code> （ <code>fico</code> 上下限均值）                           | 同一评分的区间表示合并为一个值   |
| <code>`credit_history_years`</code> （信用历史长度，中位数 15 年）                        | 信用历史越长通常越可信       |
| <code>`installment_income_ratio`</code> （月供收入比）                              | 衡量"还不起"的压力        |
| <code>`grade_dti_inter`</code> 、 <code>`interest_loan_inter`</code> （2 个交互项） | 计量思想：高风险因素叠加，风险更高 |

### 5.4 特征选择（60 列 → 保留 38 个特征）

过滤法（与违约的相关系数）+ 嵌入法（随机森林特征重要性）交叉判断，删除"重要性 < 均值 1/10"的弱特征。被删的主要是独热编码产生的小类别列；``purpose`` 的独热列全部被删，说明贷款用途单独看对违约预测贡献很小（已被利率、等级等特征吸收）——这是特征选择给出的客观结论。

特征重要性 Top 5（随机森林）：

| 特征                           | 重要性   |
|------------------------------|-------|
| <code>subGrade_enc</code>    | 0.188 |
| <code>interestRate</code>    | 0.134 |
| <code>grade_enc</code>       | 0.109 |
| <code>term</code>            | 0.069 |
| <code>grade_dti_inter</code> | 0.045 |

### 5.5 防数据泄漏（全程底线）

目标编码、标准化、缺失填充等一切"从数据学到的规则"只从训练集学习、再套用到测试集，保证测试集（以及未来的新客户）对模型是真正"没见过"的。若有泄漏，最终评估会虚高、实际业务会翻车。

### 5.6 方法取舍：文档里列了、我们评估后没用上的方法

任务文档在本阶段还列了另外几种方法。逐条评估后没有全部采用，原因都一样：这些方法在这份数据上要么效果更差，要么收益配不上成本。

| 方法              | 为什么不用                                                                                                                                               |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| 最小最大归一化（MinMax） | 用"最大值 - 最小值"当分母，对极端值极敏感：收入最大 1,099 万，99.9% 的人被压到贴近 0 的一小段，差异等于抹平。改用 <code>log1p + z-score</code> ，长尾压缩、相对距离保留。                                      |
| 分箱（等频 / 等距切档）   | ① <code>`grade`</code> / <code>`subGrade`</code> 本身就是机构分好的风险档；② 树模型分裂时自己在找最优切分点，人工再分一遍是重复劳动；③ 风控经典的分箱是 WOE， <code>`regionCode`</code> 用的目标编码属同一族思路。 |
| 多项式特征（全量两两相乘）   | 38 个特征两两相乘会膨胀到 700 多个，56 万样本下                                                                                                                       |

|                 |                                                                                                 |
|-----------------|-------------------------------------------------------------------------------------------------|
|                 | 极易过拟合。改为只造业务上最该交互的两组：`grade × dti`、`interestRate × loanAmnt`，其中 `grade_dti_inter` 进了重要性 Top 5。  |
| 包装法 RFE（递归特征消除） | 每轮只丢一个特征、重训一次，56 万行 × 38 特征跑一轮要几十分钟到几小时；而过滤法加嵌入法两路交叉已把 60 列筛到 38 个，收益配不上成本。5.7 节用更便宜的方式给了同样的答案。 |

## 5.7 特征数量实验：38 个都得上吗

上述筛选解决了"该不该留"，业务方还会问：38 个特征是不是都得采集？少放几个会掉很多吗？（字段越少，采集、清洗、部署的成本越低。）为此做一次"逐步加特征、每次重训同一个 LGB 模型"的实验：按 W2 随机森林重要性从高到低排序，依次只喂前 N 个特征，比较验证集 AUC。为避免污染最终评估，本实验只看训练集与验证集，测试集全程不参与。

这一步做在 W4 收尾阶段（`scripts/W4/feature\_count\_auc.py`），排在特征定稿与调参之后，属于事后检验：特征排序取自 W2，重训用的是 W3 调好的那套参数，结论只用来回答"采集口径能不能精简"，没有反过来改动最终模型。

| 使用特征数        | 训练集 AUC | 验证集 AUC | 相比满血模型的差距 |
|--------------|---------|---------|-----------|
| Top 3        | 0.6974  | 0.7005  | -0.0289   |
| Top 5        | 0.7097  | 0.7074  | -0.0220   |
| Top 10       | 0.7221  | 0.7131  | -0.0163   |
| Top 15       | 0.7317  | 0.7205  | -0.0089   |
| Top 20       | 0.7385  | 0.7253  | -0.0041   |
| Top 25       | 0.7414  | 0.7284  | -0.0010   |
| Top 30       | 0.7419  | 0.7284  | -0.0010   |
| Top 38（最终模型） | 0.7432  | 0.7294  | —         |

怎么读这张表：

3 个特征拿到 96% 的性能（0.7005 / 0.7294），这 3 个是 `subGrade\_enc`、`interestRate`、`grade\_enc`，与 4.1 节"等级是最强分层变量"对上。

20 个特征达到满血模型的 99.4%（0.7253），补齐剩下 18 个只多换来 +0.0041；25 个之后 25 → 30 一点没涨，25 → 38 也只涨 0.0010。

训练集 AUC 一路涨到 0.7432，验证集早早走平：后加的特征多在帮模型记训练集，所以不必把 60 列全塞进去。

业务结论：最终模型保留全部 38 个特征，它们已经在数据集里，多留着不增加线上成本。如果将来要精简采集口径（某些字段需要客户额外授权或人工录入），砍到 Top 20-25 个特征、AUC 损失不到 0.005，是有数据支撑的可行方案。

## 6. 基线模型构建与对比（W2）

### 6.1 数据划分（721）

按任务文档要求：训练 70% / 验证 20% / 测试 10%，分层抽样（每份违约率都约 20%）。三份数据的分工只有一句话：模型自己算出来的数字只用训练集，人做选择在验证集上试，对外公布的数字用测试集、而且只揭一次（9.5 节的校准、10.1 节的选阈值都按这条做）。

| 数据集 | 人数      | 用途                        |
|-----|---------|---------------------------|
| 训练集 | 558,948 | 训练模型、调参（5 折交叉验证在这里面切）     |
| 验证集 | 159,700 | 模型对比与选参、拟合校准、选审批阈值、特征数量实验 |
| 测试集 | 79,850  | 只出最终成绩与金额，不做任何选择          |

## 6.2 三个基线模型

| 模型          | 一句话解释        | 在本项目中的角色    |
|-------------|--------------|-------------|
| 逻辑回归 [11]   | 线性模型，输出违约概率  | 线性基准，可解释性最强 |
| 随机森林 [4]    | 多棵决策树投票      | 非线性基准，抗过拟合  |
| XGBoost [3] | 梯度提升树（树逐个纠错） | 树模型主力，预期最强  |

类别不平衡处理：三模型均设置 `class\_weight='balanced'`（LGB 用 `scale\_pos\_weight`），让模型更重视少数类（违约者）。

## 6.3 评估指标怎么读

准确率 **accuracy**：判对的比例。不平衡数据下会骗人（全猜正常也有 80%）。

精确率 **precision**：模型说“违约”的人里，真违约的比例。误伤多则低。

召回率 **recall**：真违约的人里，被抓到的比例。漏网多则低。

F1：精确率与召回率的调和平均，两个都要时看它。

AUC：随机抽一对（违约 vs 正常），模型把违约者排在正常者前面的概率。排序能力，与阈值无关，本报告最核心指标。

## 6.4 验证集三模型对比（阈值 0.5）

| 模型      | 准确率    | 精确率    | 召回率    | F1     | AUC    |
|---------|--------|--------|--------|--------|--------|
| 逻辑回归    | 0.6582 | 0.3241 | 0.6572 | 0.4341 | 0.7189 |
| 随机森林    | 0.6507 | 0.3208 | 0.6723 | 0.4343 | 0.7198 |
| XGBoost | 0.6559 | 0.3259 | 0.6785 | 0.4403 | 0.7276 |

结论：XGBoost 最优（AUC 0.7276）；三个模型 AUC 都在 0.72 左右，结论互相印证。图见 `outputs/W2/roc\_curves.png`、`outputs/W2/pr\_curves.png`。

## 7. 模型优化（W3）

### 7.1 调参策略：为什么用随机搜索

网格搜索：正式空间是 6 个参数 × 每个 3 个取值（ $3^6 = 729$  组），每组还要跑 5 折，共 3,600 多次训练，55.9 万行的训练集上时间不现实；

贝叶斯优化（TPE）：根据前几次的成绩优先试“更可能出好成绩”的区域，理论上更省次数；

随机搜索：从参数空间随机抽固定组数（每个模型 8 组），实现与复现成本最低。

结论：我们用“2 个模型 × 3 种方法”的全组合等预算对照验证了这一点——同一模型内三种方法的最优成绩完全一致（极差不到 0.0002），差别只在试错效率，因此正式调参采用随机搜索。

### 7.2 交叉验证

`StratifiedKFold(5)`：训练集切 5 份轮流当验证，每折违约率保持 20%；16 组参数的 5 折平均 AUC 标准差  $\leq 0.00071$ ，评分稳定可靠。

### 7.3 模型集成

投票法（soft voting）：LR/RF/XGB/LGB 四个模型各给一个违约概率，取平均。

堆叠法（stacking）：把四个模型的预测概率当“新特征”，再训练一个逻辑回归当“教练”综合判断（内部用 5 折交叉验证防止教练背答案）。

### 7.4 优化前后对比（验证集）

| 模型     | 准确率    | 精确率    | 召回率    | F1     | AUC    |
|--------|--------|--------|--------|--------|--------|
| LR(基线) | 0.6582 | 0.3241 | 0.6572 | 0.4341 | 0.7189 |

|              |        |        |        |        |        |
|--------------|--------|--------|--------|--------|--------|
| RF(基线)       | 0.6507 | 0.3208 | 0.6723 | 0.4343 | 0.7198 |
| XGB(基线)      | 0.6559 | 0.3259 | 0.6785 | 0.4403 | 0.7276 |
| XGB(优化)      | 0.6636 | 0.3301 | 0.6672 | 0.4417 | 0.7286 |
| Voting(集成)   | 0.6601 | 0.3280 | 0.6708 | 0.4405 | 0.7276 |
| LGB(优化)      | 0.6566 | 0.3271 | 0.6825 | 0.4422 | 0.7294 |
| Stacking(集成) | 0.6553 | 0.3267 | 0.6860 | 0.4426 | 0.7298 |

三点结论：① 调参有效但幅度小（AUC 提升约 +0.002）——参数优化的是细节，改变不了特征的信息量；② 集成略高于单模型但接近——四个队员里三个是“树亲戚”，观点太像，互补空间有限；③ 所有模型 AUC 都在 0.72–0.73，结论稳健、不依赖某一个模型。

## 7.5 最终模型选择

选 LGB(优化)[2]（验证集 AUC 0.7294，与 Stacking 的 0.7298 几乎无差）。理由：LGB 训练快（实测 5.6 秒，Stacking 371 秒、约 66 倍）、可用 SHAP 做业务解释（Stacking 的“教练”是逻辑回归，无法直接做树级解释）、部署简单。业务交付，可解释性优先于 0.0004 的 AUC 差距。

## 8. 模型解释：SHAP 特征归因（W3）

对象：LGB(优化)，训练集随机抽 5 万行计算（SHAP 方法见 [5]）。Top 10 见下表（完整 38 个见 `outputs/W3/shap\_feature\_importance.csv`，图见 `outputs/W3/shap\_summary.png`）：

| 特征                       | 归因强度  | 含义              |
|--------------------------|-------|-----------------|
| subGrade_enc             | 0.335 | 子等级（贷款细分档位）     |
| term                     | 0.165 | 贷款期限            |
| homeOwnership_1          | 0.122 | 房屋所有权           |
| grade_enc                | 0.099 | 贷款等级            |
| fico_avg                 | 0.091 | FICO 信用分（W2 新建） |
| regionCode_enc           | 0.086 | 地区编码            |
| n14                      | 0.083 | 匿名行为特征          |
| interestRate             | 0.077 | 贷款利率            |
| dti                      | 0.077 | 债务收入比           |
| installment_income_ratio | 0.076 | 月供收入比（W2 新建）    |

业务解读：模型“看人先看贷款条件”——子等级 + 期限 + 等级合计约占一半归因；FICO、dti、月供收入比等“还款能力”特征紧随其后，符合风控直觉。W2 新建的 `fico\_avg`、`installment\_income\_ratio` 进入前 10，证明特征工程有效。匿名特征 n14 等有信号但无法业务解释，只能黑箱使用。

## 9. 最终模型评估（W4，10% 测试集）

用从未参与任何建模决策的 79,850 人做最终确认。先复测验证集（与 W3 的 LGB 行完全一致，证明切分无偏差），再出最终成绩单。

### 9.1 测试集成绩单（阈值 0.5）

| accuracy | precision | recall | F1     | AUC    | KS     |
|----------|-----------|--------|--------|--------|--------|
| 0.6560   | 0.3254    | 0.6749 | 0.4390 | 0.7249 | 0.3269 |

KS 0.3269 是违约者与正常人的分数分布拉开的最大距离，风控常用的门槛是 0.30，最大间距出现在分数 0.503 附近（`outputs/W4/ks\_summary.csv`）。

排序能力在名单头部更明显。审批人力有限，真正被逐条看的是分数最高的那一批人（`outputs/W4/head\_capture.csv`）：

| 分数最高的 | 人数 | 命中违约人数 | 捕获率 | 相对随机 | 这批人自己的违 |
|-------|----|--------|-----|------|---------|
|-------|----|--------|-----|------|---------|

|       |        |       |       |        | 约率    |
|-------|--------|-------|-------|--------|-------|
| 前 5%  | 3,992  | 2,174 | 13.6% | 2.73 倍 | 54.5% |
| 前 10% | 7,985  | 3,839 | 24.1% | 2.41 倍 | 48.1% |
| 前 20% | 15,970 | 6,535 | 41.0% | 2.05 倍 | 40.9% |
| 前 30% | 23,955 | 8,693 | 54.6% | 1.82 倍 | 36.3% |

捕获率指这批人抓到的违约者占全部违约者（15,929 人）的比例。前 10% 的 7,985 人里近一半真的违约，是整体违约率 19.9% 的 2.4 倍。KS 与捕获率只看排序，与概率刻度无关，校准前后数值一样。

## 9.2 混淆矩阵

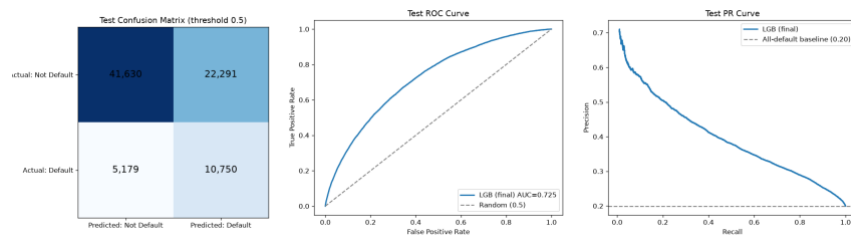


图 2 | 测试集上的混淆矩阵（左）、ROC 曲线（中）与 PR 曲线（右），阈值 0.5，79,850 人

在 79,850 人中：模型判"违约"约 3.3 万人，其中约 1.08 万人真违约（精确率 0.325）；真违约的 1.6 万人中抓到约 1.08 万（召回率 0.675）。误伤好人是这种"抓坏人"任务的固有代价，靠人工复审消化。

## 9.3 ROC 与 PR 曲线

排序质量见上图后两幅：

ROC：曲线远离对角线，AUC 0.725，排序能力明显强于随机（0.5）。

PR：曲线明显高于"全猜违约"基准线（0.20），说明抓违约者的效率远好于无脑全猜。

## 9.4 泛化能力分析（训练/验证/测试三集对比）

| 数据集 | 准确率    | 精确率    | 召回率    | F1     | AUC    |
|-----|--------|--------|--------|--------|--------|
| 训练  | 0.6633 | 0.3352 | 0.6999 | 0.4533 | 0.7432 |
| 验证  | 0.6566 | 0.3271 | 0.6825 | 0.4422 | 0.7294 |
| 测试  | 0.6560 | 0.3254 | 0.6749 | 0.4390 | 0.7249 |

泛化结论：训练集 AUC 只比测试集高 0.018（0.7432 vs 0.7249），三集指标几乎贴着走——模型没有明显过拟合，学的是真实规律而非背题；验证集结论能原样搬到测试集，调参没有"泄漏进测试集"的嫌疑。

## 9.5 概率校准：让"概率"能直接算钱

9.1–9.4 证明模型的排序能力可靠（AUC 0.725），但输出概率的数值本身不可信：测试集上模型给所有人的平均违约概率是 0.4495，而实际违约率只有 0.1995——概率整体偏高约 125%；阈值 0.5 时 41.4% 的人会被判违约，真实只有约 20%。

原因：训练时为应对类别不平衡（违约率 20%）做了加权（`scale\_pos\_weight`），让模型优先抓少数类，代价就是把概率"语气说重了"。

处理办法是概率校准：用一批已知结果的数据，学一个把 0.45 修正到 0.20 附近的映射（`scripts/W4/probability\_calibration.py`）。两种主流方法先在验证集内部对半比较（一半拟合、一半评分），选定后再应用到测试集；测试集全程不参与拟合：

| 方案                | Brier（概率误差，越小越好） | AUC（排序能力） | 平均预测概率 |
|-------------------|------------------|-----------|--------|
| 未校准               | 0.2106           | 0.7249    | 0.4495 |
| Isotonic（保序回归）    | 0.1425           | 0.7248    | 0.1993 |
| Platt（Sigmoid 校准， | 0.1424           | 0.7249    | 0.1993 |

|            |   |   |        |
|------------|---|---|--------|
| 采用)        |   |   |        |
| (参考：实际违约率) | — | — | 0.1995 |

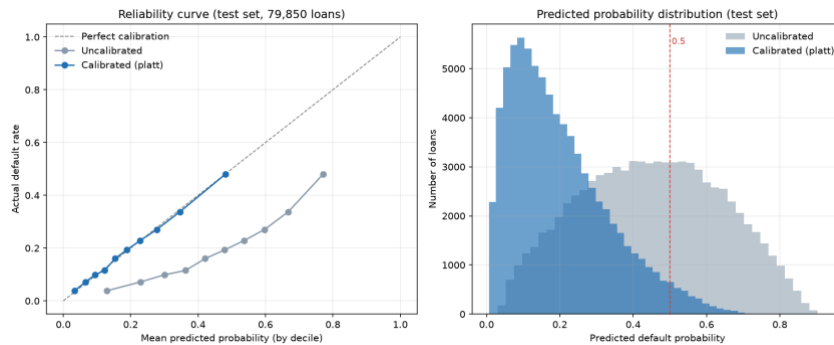


图 3 | 概率校准曲线：校准后平均概率 0.1993，贴近实际违约率 0.1995

校准后平均概率 0.1993，与实际违约率 0.1995 几乎完全一致；概率预测误差（Brier 分数）从 0.2106 降到 0.1424。

AUC 保持不变（0.7249）：校准只修正概率的数值标度，不改变排序能力。两种方法几乎打平（测试集 Brier 0.1424 vs 0.1425），选实现更简单、小样本下更稳的 Platt（校准方法见 [6][7][8]，概率评估指标与神经网络校准的系统讨论见 [9][10]）。

两把刻度一一对应，挑线结果不变：校准严格单调（按原始概率排序后校准概率下降 0 次，Spearman 相关 1.0000000000），原始轴的  $p_{\text{raw}} \geq 0.553$  与校准轴的  $p_{\text{cal}} \geq 0.2406$  拒的是逐行相同的 25,693 人。两把刻度上的最优线是同一条，只是数字不同；数字不能跨轴直接搬——把 0.553 拿到校准轴上用，只拒 1.7% 的人。

为什么这一步重要：校准前，模型说“概率 0.6”只能理解为“相对更危险”；校准后概率是违约率的真实估计，可以直接乘金额算期望损失，这是第 10 节决策框架的地基。校准后概率集中在 0–0.3 区间，原先“阈值 0.5”不再对应 41% 的拒绝率，业务切割线要按金额账重新定。

#### 9.6 时间外验证：拿过去训练，给未来打分

9.1 到 9.5 的成绩都出自随机切分：同一批放款随机分到训练和测试，两边行情一样，成绩会偏乐观。真实上线只能用过去的训练数据，给未来上门的客户打分。按放款日期再切一次（`scripts/W4/oout\_validation.py`）：训练用 2016 年底之前的 664,968 笔，测试用 2017-01 到 2018-06 的 124,550 笔。2018-07 之后的 8,980 笔没放进来，那段违约率只有 7.5%，是真变好还是坏账还没暴露，从数据里分不清。

| 口径             | 样本数     | 违约率   | AUC    | KS     | 前 10% 捕获率 | 阈值 0.5 拒绝率 | 阈值 0.5 精确率 |
|----------------|---------|-------|--------|--------|-----------|------------|------------|
| 随机切分（对照）       | 124,550 | 19.9% | 0.7279 | 0.3300 | 24.1%     | 41.9%      | 0.325      |
| 时间外（2017-01 起） | 124,550 | 22.3% | 0.7184 | 0.3190 | 21.7%     | 43.9%      | 0.348      |

排序能力掉了一点，没伤到根：AUC 从 0.7279 到 0.7184，KS 从 0.3300 到 0.3190，仍明显强于随机；前 10% 捕获率 21.7%，远高于随机挑的 10%。

行情变化的影响更大：时间外这段违约率 22.3%，比随机切分高 2.3 个百分点；同一条 0.5 的线，拒人比例从 41.9% 升到 43.9%，精确率从 0.325 升到 0.348。

概率刻度会漂：时间外的 10 个分箱里，预测概率每一档都高于实际违约率（最低一档预测 0.120、实际 0.038；最高一档预测 0.788、实际 0.483，`outputs/W4/oout\_calibration.csv`）。第 10 节的钱账建立在概率数值上，那条轴要定期用最近到期的贷款重新标定；只看排序的阈值可以放宽心。

#### 10. 风险阈值与定价空间（业务决策）

## 10.1 固定阈值对照表（放贷方案① 单线扫描）

模型输出的是概率，机构需要定一条"多少分以上算违约"的线（阈值）。下表把几条候选线放在同一个测试集上，人数账和钱账一起给（净息差 6%、回收率 30%，概率用校准后的值；金额单位万元，来源 ``outputs/W4/threshold_sweep.csv``）：

| 阈值   | 拒绝率   | 精确率   | 召回率   | 净增益   | 总期望利润 |
|------|-------|-------|-------|-------|-------|
| 0.30 | 75.1% | 0.245 | 0.923 | 1,566 | 3,564 |
| 0.40 | 58.7% | 0.281 | 0.828 | 3,126 | 5,119 |
| 0.45 | 50.1% | 0.302 | 0.759 | 3,677 | 5,737 |
| 0.50 | 41.4% | 0.325 | 0.675 | 3,980 | 6,155 |
| 0.55 | 32.7% | 0.353 | 0.579 | 4,144 | 6,340 |
| 0.60 | 24.5% | 0.386 | 0.475 | 4,019 | 6,237 |
| 0.65 | 17.2% | 0.428 | 0.369 | 3,752 | 5,827 |
| 0.70 | 11.0% | 0.474 | 0.262 | 3,080 | 5,130 |

两列口径不同，别混着看：净增益 = 避免的坏账 - 放弃的利息收入（相对"全批放款"，全批为 0）；总期望利润 = 给每笔被批准的贷款算  $(1-p) \times \text{净息差} \times \text{期限} - p \times (1 - \text{回收率})$  后求和（与 10.2 节同一把尺子，全批放款为 2,085 万元）。

举例对账：0.553 处按净增益算是比全放多 4,157 万元，同一处换成期望口径是比全放多 4,259 万元（6,344 - 2,085）。差 102 万元来自被拒人群的实际违约率 35.5%，比模型估的 36.0% 略低。线上没有答案，规则与落地建议只用期望口径（10.2-10.3 节）；净增益只在历史数据上用来挑线。

这条曲线不需要校准：概率在 10.1 节只负责排名，金额那两栏数的是答案（被拒的人里谁真违约、谁其实正常）。10.2 节把概率乘进了金额，概率的数值直接决定每笔账的正负，才要求先用 9.5 节的校准把刻度修正。

曲线形状是"两头都亏、中间最高"：线拧到 0.30 时净增益掉到 1,566 万元（误伤太多，放弃的收入超过避免的坏账）；拧到 0.70 只剩 3,080 万元（该拒的没拒）。

怎么选：

阈值低（0.3）= 宁杀错不放过：抓到 92% 的违约者，但 3/4 的申请被拦下，误伤多。适合坏账损失极高、风险偏好保守的时期。

阈值高（0.7）= 只拦最危险的：被抓的人里近一半是真违约（精确率 0.474），但漏掉大量违约者。适合放量增长期。

起手建议 0.55：把线从 0.50 挪到 0.55，少拒 6,944 人，期望利润从 6,155 万升到 6,340 万元。

落地方式：线以下的直接放、贴着线的进人工复审、更上面的直接拒（10.3 节按拆线的两条线算人数，用单线要按所选阈值换算），模型当排序工具而不是最终裁判。

这条线该在哪里选：上面的最优线（0.553）是在测试集上扫出来的，而测试集理论上只该用来验一次。改用验证集选线、测试集只跑一次的口径核对（``scripts/W4/threshold_select_on_val.py``）：

在验证集上选出的最优阈值是 0.560，把它原样搬到测试集跑一次：拒绝 30.9%、期望利润 6,342 万元；

测试集上细扫（0.001 步长）的最优点在 0.553、6,344 万元（``outputs/W4/threshold_sweep.csv``），与验证集口径差 2 万元，"多赚 187 万元"不是调参调出来的；

验证集上 0.54-0.58 是一段平台区（利润都在最优值的 99.5% 以内），业务可以在这段区间里按自己的风险偏好挑，不必纠结小数点。

## 10.2 从"拍一条线"到"按笔算账"：期望损失框架（放贷方案② 逐笔期望定线）

结论：中性口径（净息差 6%、回收率 30%）下，逐笔算账解出来两条线——3 年期 0.50、5 年期 0.62（模型原始输出轴）。全批期望利润 6,611 万元，比最优单线（0.553、6,344 万元）多 267 万元。代价是审批系统要按期限出两张评分表。

为什么这么算：固定阈值是"一刀切"，概率过线就拒，不管金额多大、期限多长、利率多高。一笔贷款值不值得放，要看金额、期限、利率（赚多少）和回收率（亏多少），把这几件事合成一笔账

（``scripts/W4/expected_loss_framework.py``）：

每 1 元贷款的期望利润 =  $(1 - p) \times \text{净收益率} \times \text{期限} - p \times (1 - \text{回收率})$

其中  $p$  必须用校准后的概率（9.5 节），否则概率虚高、账全算错。决策规则：期望利润  $> 0$  批准，否则拒绝——它等价于给每笔贷款算一条自己的隐含阈值，赚头厚的能容忍更高风险、利差薄的更严格。中性口径把净息差统一按 6% 算，赚头的差别就只剩期限：5 年期容忍到 0.3000、3 年期到 0.2045（校准轴）。金额在公式两边同时出现，不改变判定；利率被这个统一值吸收，没有进规则——要让它进规则，得换成逐笔净息差，即合同利率减去资金成本与运营成本（见本节末尾的声明）。本批数据只有 3 年期与 5 年期两种贷款，这条规则落地下来就是两条线。

下表统一用“净息差 6%、回收率 30%”口径评价经济后果（保证各策略可比；参数为演示假设，需机构财务口径确认）：

| 策略                     | 被拒占比  | 拦截率（真违约被拒） | 被拒者中真违约占比 | 总期望利润    |
|------------------------|-------|------------|-----------|----------|
| 全放（不筛选）                | 0%    | 0%         | —         | 2,085 万元 |
| 固定阈值 0.5               | 41.4% | 67.5%      | 32.5%     | 6,155 万元 |
| 期望损失框架 · 利息全额（最乐观）     | 1.8%  | 4.6%       | 52.4%     | 2,670 万元 |
| 期望损失框架 · 净息差 6%（中性，推荐） | 35.2% | 59.8%      | 33.9%     | 6,611 万元 |
| 期望损失框架 · 净息差 3%（最保守）   | 65.3% | 86.4%      | 26.4%     | 4,983 万元 |
| 期望损失框架（未校准概率，对照）       | 87.7% | 97.4%      | 22.2%     | 2,110 万元 |

表里“期望损失框架 · 净息差 6%（中性，推荐）”那一行是推荐口径；利息全额与净息差 3% 两行是同一框架换收益假设的结果，用来看结论有多依赖参数。

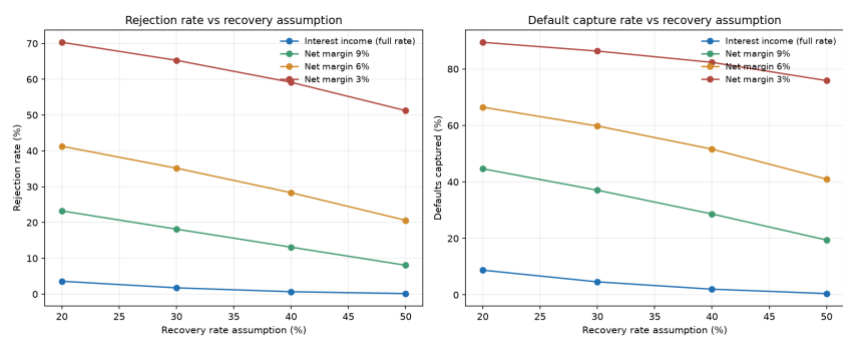


图 4 | 期望损失敏感性：同一框架下，收益口径比回收率影响大得多

怎么读：

中性情景下，框架比固定阈值“少拒多赚”：拒绝 35.2%（低于固定阈值 41.4%），期望利润反而更高（6,611 vs 6,155 万元）。原因：框架不追求“多抓坏人”，而是“每笔账为正”，主动放过那些“利息收入能覆盖预期损失”的高风险客户。

收益口径是最大敏感源：同样的框架，收益假设从“利息全赚”换成“净息差 3%”，拒绝率从 1.8% 涨到 65.3%。最终切割线必须由机构真实财务口径决定，不能拍脑袋。

校准是前提：不校准概率直接套框架，拒绝率虚高到 87.7%、期望利润只剩 2,110 万元；换成校准概率是 35.2% / 6,611 万元。这 4,502 万元全部来自把刻度修准，规则形式、金额、期限都没变。

期限只决定每笔的线画在哪： $p^* = \text{净息差} \times \text{期限} \div (\text{净息差} \times \text{期限} + 1 - \text{回收率})$ 。中性口径下按校准轴解出 3 年期 0.2045、5 年期 0.3000，对应原始输出轴的 0.50 与 0.62——两条线的位置是这样算出来的，不是扫出来的。

这笔账到底看了什么：逐笔进公式的只有三样——模型算出的违约概率、贷款金额、期限。而资金成本、运营成本、这笔贷款将来能催回多少，数据集里都没有，故只能按 6% 和 30% 的机构级假设统一处理。所以现有数据撑得住的是“该不该放”——按期限两条线、贴线的交人工；

撑不住的是"能赚多少"的精确数字，也给不出个性化定价。要再往前一步，缺的不是模型，是机构的财务成本口径和催收回收数据。

诚实声明：这是演示性测算，不构成利润结论。净收益率没有逐笔按各自的合同利率扣减资金成本与运营成本，而是统一取 6%（中性）或 3%（保守）——这样做抹平了逐笔利率差异，本批利率在 5.31%–30.99% 之间；违约损失按本金减回收率计，回收率固定 30%。这些参数都是假设值，真实落地需用机构财务成本数据重新标定。

### 10.3 落在线上的人交人工：8,697 人的复审带（放贷方案② 执行细节）

结论：拆线方案判拒的 28,078 人不该一刀切拒掉。里面贴着线的 8,697 人（线上方 20% 以内）转人工复审，更上面的 19,381 人机器直接拒，线以下的 51,772 人机器直接放。

为什么这么分：10.2 的框架给每笔贷款算出一条自己的线，线附近的人结论最不稳，模型概率稍有偏差判定就翻面，而这一段里很可能压着被误拒的好客户。线以下不设复审带，那里模型认为放比拒划算，人工优先用在会被误拒的一侧（`scripts/W4/review\_band.py`）。带宽取线上方 20% 以内（校准轴：3 年期 0.2045 到 0.2455，5 年期 0.3000 到 0.3600）：

| 分工               | 人数     | 占测试集  | 违约率（模型预估 / 事后实际） |
|------------------|--------|-------|------------------|
| 直接放行（线以下）        | 51,772 | 64.8% | 12.1% / 12.4%    |
| 人工复审（线上方 20% 以内） | 8,697  | 10.9% | 25.9% / 25.4%    |
| 直接拒绝（更上面）        | 19,381 | 24.3% | 38.2% / 37.8%    |

三段里模型的预估违约率依次是 12.1%、25.9%、38.2%，事后实际的违约率是 12.4%、25.4%、37.8%，两边对得上：校准后的刻度在这批人上是准的，这个分法不靠运气。

复审带占被拒 28,078 人的 31.0%，里面 6,488 名是正常客户。一刀切拒掉，丢的就是这批人的生意：这 6,488 人放出去，息差有 2,188 万元。

反过来，这 8,697 人不挑不拣全放，按模型口径期望利润 -295 万元。复审带的价值就在这两头之间摆动，转人工就是把它往 +2,188 万元那头拉。

带宽多宽属于审批政策，不是模型算出来的：放宽到线上方 30%，复审人数从 8,697 涨到 12,381（占测试集 15.5%）。

### 10.4 延伸测算：低风险客户的让价空间

10.1 到 10.3 回答的是"谁不放款"。同一根校准后的概率轴还能算另一个问题：风险低的人，价格便宜到什么程度还不亏。这一节是延伸测算，不是定价方案——数据集里没有授信额度、留存率和同业价格，算得出让价上限，算不出该不该让。

口径与 10.2 节一致（净息差 6%、回收率 30%）。把期望利润置零反解，得到每个风险档的保本净息差，也就是这一档要不亏钱，每年至少要赚回多少：

保本净息差 =  $p \times (1 - \text{回收率}) \div [(1 - p) \times \text{期限}]$

p 用校准后概率（9.5 节）。保本净息差离 6% 的假设有多远，就是这一档还能让出多少价（

`scripts/W4/pricing\_headroom.py`）：

| 校准概率档  | 人数     | 实际违约率 | 平均利率   | 保本净息差 | 让价空间     | 每万元期望利润  |
|--------|--------|-------|--------|-------|----------|----------|
| <2%    | 1,265  | 2.1%  | 5.87%  | 0.4%  | 5.6 个百分点 | 1,665 元  |
| 2–5%   | 6,329  | 4.1%  | 7.06%  | 0.9%  | 5.1 个百分点 | 1,500 元  |
| 5–10%  | 14,026 | 8.0%  | 9.44%  | 1.9%  | 4.2 个百分点 | 1,208 元  |
| 10–20% | 24,735 | 14.9% | 12.27% | 3.7%  | 2.3 个百分点 | 723 元    |
| 20–40% | 25,985 | 27.6% | 15.87% | 7.4%  | 无        | -313 元   |
| >40%   | 7,510  | 48.8% | 20.72% | 14.4% | 无        | -1,984 元 |

风险越低，账越经得起让价：概率低于 2% 的 1,265 人保本净息差只要 0.4%，离 6% 的假设还有 5.6 个百分点；2–5% 档还有 5.1 个百分点，这两档付的利率是 5.87% 和 7.06%，风险溢价收得比代价厚。

高风险档没有让价空间：20-40% 档保本净息差 7.4%，已经高过 6% 的收益假设，每万元期望利润 -313 元；40% 以上档要 14.4%，每万元 -1,984 元。这两档要么按 10.2 节的线拒掉，要么反向上浮利率。

可以拿去谈的量级：校准概率低于 5% 的 7,594 人（占测试集 9.5%，人均放款 13,749 元），利率每下浮 1 个百分点，全生命周期让价 316 万元；下浮 2 个百分点是 632 万元。这批人自身贡献期望利润 1,593 万元，让价吃掉其中两成；划不划算取决于价格弹性，数据里没有留存率和同业价格，上线前要用 A/B 补。

额度只能给方向，口径也要提醒：数据集只有申请金额 `loanAmnt`，没有授信额度、用信率，提多少给不出数字；能说的是单位风险成本——低风险档每贷出 1 万元赚 1,500-1,665 元，20% 以上两档每万元是负的，加额度等于放大亏损。表中“让价空间”是相对 6% 假设的余量，不是建议定价，真实幅度要用机构的资金成本和目标收益重算。

## 11. 业务建议汇总

1. 定位：模型适合做风险排序初筛 + 辅助人工审核，AUC 0.725、KS 0.3269 尚不足以支撑全自动放款；审核人力优先投入分数最高的那批人（前 10% 抓到 24.1% 的违约者）。
2. 第一道防线看贷款条件：子等级、期限、等级、利率、FICO 是模型最看重的信息，风控规则引擎可以围绕它们设置硬性门槛（如特定等级加高利率的组合直接降额）。
3. 阈值按经营目标调，起手线建议 0.55：把线从 0.50 挪到 0.55，少拒 6,944 人，期望利润从 6,155 万升到 6,340 万元；改用验证集选线，则 0.560 对应 6,342 万元，两条路只差 2 万元。验证集上 0.54-0.58 是一段平台区，保守期往低端靠、扩张期往高端靠都能接受（10.1 节）。审批系统能拆线就按期限设两条，上限 6,611 万元（10.2 节）。
4. 给误伤留出口（放贷方案② 执行细节）：被判违约的人里约 1/3 才是真违约，其余是误伤；把线附近 20% 以内的 8,697 人交人工复审、两头由机器判，是 10.3 节给出的落地分法。
5. 上线后要重做的两件事：概率轴的重标定（9.6 节的时间外漂移要用最近到期的贷款修正），以及按月滚动收集新放款表现、重训模型并监控 AUC 与 KS 衰减。
6. 方案② 怎么和改造成本比：测试集 79,850 笔（放款额 11.53 亿元）上 ② 比最优 ① 多 267 万元，摊成“每 100 亿元年放款额”约 2,319 万元/年（净息差压到 3% 时缩到约 787 万元）。这个数高过“一次性改造费 ÷ 摊销年限 + 年运维费”一个量级就值得改，只够打平先维持 ①。前置条件还有三条：审批系统能否按期限出两张评分表、财务能否给真实净息差与回收率、有没有人维护校准层。
7. （延伸）定价侧的让价空间：校准概率低于 5% 的 7,594 人（占测试集 9.5%）保本净息差不到 1%，利率每下浮 1 个百分点让价 316 万元，约占这批人自身期望利润（1,593 万元）的两成。20% 以上两档保本净息差 7.4%、14.4%，高过 6% 的收益假设，没有让价空间。这一条只是让价上限的测算，成不成取决于价格弹性，数据集里没有留存率和同业价格，落地前用 A/B 补（10.4 节）。

## 12. 局限性与后续工作

本节把当前模型的局限按来源分成四类，逐条写清四件事：卡在哪里、怎么解、需要什么条件、怎么算解决。12.1 针对现有局限，12.2 是投入递增的增强清单，12.3 给落地顺序。

### 12.1 现有局限的解决路径

#### 数据层

1. 违约率 20% 高于真实客群（消费信贷客群通常低于 5%）

卡点：样本来自历史放款客户，本身已过一轮筛选，概率水平会整体偏高。

解法：拿到目标客群违约率  $n^*$  后，在赔率上做平移。校正系数  $k = [n^*/(1-n^*)] \div [n/(1-n)]$ ， $n$  是样本违约率（19.95%）；校正后概率  $p' = k \cdot p / [k \cdot p + (1-p)]$ 。这一步只改概率刻度，不改排序：样本里 0.20 的人，换到 3% 违约率的客群口径下约落到 0.03。若还能拿到客群结构，再按“信用等级 × 期限”分层重加权。

需要：目标客群违约率与结构占比（机构内部数据）。

验收：10 档分箱的平均预测概率与实际违约率偏差控制在 2 个百分点以内（现有校准流程可直接复用）。

2. testA 无标签，线上集无法直接评估

解法：短期用分布漂移做代理监控（PSI、特征均值与分位数漂移）；中期做影子模式，线上照常打分但不干预决策，等 3 至 6 个月账龄成熟后回收真实表现。

需要：线上打分链路账龄数据。

验收：漂移告警能提前一个月触发；影子样本上的 AUC 与 KS 同离线结果的差距在可接受区间。

### 3. 被拒客户的反事实不可观测（选择性标签）

卡点：被拒的人后来还不上不会被记录，“避免的坏账”始终是估计值。

解法：先用拒绝推断（parcelling、fuzzy augmentation）估计被拒人群的违约率；再在预计损失可控的额度内做小流量探索性放款，用逆概率加权（IPW）修正选择偏差。

需要：审批政策允许的探索额度与合规确认。

验收：加入拒绝推断后全量申请的排序指标方向稳定；探索样本的实际违约率落在预估区间内。

### 4. 定价侧授信额度、用信率与价格弹性

解法：接入授信额度与用信数据做额度分层；用历史调价记录或 A/B 实验估价格弹性。

验收：让价 1 个百分点带来的申请量提升能否覆盖息差损失（10.4 节的让价空间表按这个口径扩展）。

特征层

### 5. 匿名特征 n0 至 n14 不可解释

解法：先向数据方确认口径；确认不了就做代理画像（与已知变量的相关性、SHAP 交互、分箱违约率单调性），同时用 5.7 节的逐步加特征方法量化剔除后的性能损失。

验收：给出剔除匿名特征的 AUC 损失数值，由合规判断能否接受。

模型层

### 6. 精确率有限（阈值 0.5 下 0.325）

解法：把金额写进目标函数（用 10.2 节的期望利润做样本权重或自定义损失），不只优化 AUC；对边界人群做两阶段判定；按期限与客群分层设线。

验收：同一拒绝率下期望利润高于现状，或同一期望利润下拒绝人数更少。

### 7. 集成增益有限（基学习器同质）

解法：引入异质模型（正则化线性、深度模型、图模型），或用采样与特征子集的差异造多样性。

验收：验证集 AUC 提升不低于 0.003，且通过 DeLong 与 Bootstrap 显著性检验（现有流程可直接复用）。

### 8. 时间维度只切了一刀

解法：改成 walk-forward 多次切分、滚动训练与验证，配套月度重训与监控。

验收：多期 AUC 与 KS 波动在设定阈值内（例如 AUC 衰减不超过 0.02），校准 Brier 不随账龄上升。

方法与口径

### 9. 最优阈值曾在测试集上选出

现状：已修正，10.1 节给出验证集口径（0.560），与测试集细扫最优值 0.553 只差 2 万元。

解法：固化成流程并写进模型卡：验证集选线、测试集只验一次；阈值、超参数、特征选择都不看测试集。

### 10. 金额结论依赖模拟参数（净息差 6%、回收率 30%）

解法：与财务确认口径后重算，并把单点值改成区间（净息差 3% 至 9%、回收率 20% 至 50% 的组合）。

验收：参数区间内策略排序保持稳定；排序会翻转的情形要在报告里点名。

## 12.2 增强方向（按投入递增）

| 投入         | 做什么                                                        | 依赖             | 验收                         |
|------------|------------------------------------------------------------|----------------|----------------------------|
| 短期（不需要新数据） | 代价敏感目标函数、分层阈值、复审带按期望利润排序（人审名额有限时先送价值高的）、金额结论区间化、模型卡与监控指标定义 | 现有数据与约两周工程量    | 同一拒绝率下期望利润提高；复审带的单位人审收益提升  |
| 中期（需要内部数据） | 拒绝推断与探索性放款、授信与用信数据接入、价格弹性实验、概率月度重标定流水线、特征稳定性筛选（PSI 与 IV）   | 内部数据授权，风控与财务配合 | 全量申请上的排序指标稳定；每月重标定后校准偏差超阈值 |

|            |                                                    |               |                         |
|------------|----------------------------------------------------|---------------|-------------------------|
| 长期（需要组织配合） | 外部征信数据接入、可解释性约束（单调约束与反事实解释）、自动化重训与 A/B 平台、模型风险定期审计 | 合规、IT 与外部数据采购 | 通过合规评审；线上 A/B 的增量收益统计显著 |
|------------|----------------------------------------------------|---------------|-------------------------|

### 12.3 落地顺序

1. 先用真实财务口径标定金额参数，重算第 10 节的结论，这一步不改模型。
2. 上线单一线方案（0.55），同时接入监控（PSI、KS、校准偏差）与人工复审带。
3. 拿到 3 至 6 个月账龄表现后做概率重标定与月度重训，再评估是否切换到拆线方案。
4. 内部数据到位后推进拒绝推断与价格弹性实验；外部数据接入与自动化平台放在最后。

### 13. 交付物与复现方式

| 交付物    | 位置与说明                                                                                                                                                             |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 代码     | W1 `scripts/W1/`（预处理、EDA）；W2 `scripts/W2/`（特征工程、三模型基线）；W3 `scripts/W3/`（调参、集成、SHAP）；W4 `scripts/W4/`（最终评估、概率校准、期望损失框架、特征数量、阈值扫描与验证集选线、头部区分度、时间外验证、复审带、定价让价空间、仪表板） |
| 报告与图表  | 本报告 `reports/最终项目报告.md`；分阶段报告（环境配置、数据探索+EDA、特征工程、模型评估、模型优化）；周报 `reports/W*_周报*.docx`（W1-W4）；图表在 `outputs/W1/`-`outputs/W4/`                                       |
| 仪表板    | 离线单文件 `outputs/W4/dashboard.html`，双击即开、无需环境；Streamlit 版 `scripts/W4/app_dashboard.py`，阈值、回收率、净息差可拖                                                                |
| 演示 PPT | `outputs/W4/信贷违约模型与审批线_W4 汇报.pptx`（15 页，无备注），同目录 `.pdf` 同内容                                                                                                       |
| 数据与复现  | `data/train_final.csv`、`data/testA_final.csv`；完整命令序列（含每步耗时估计）见 `README.md` 的“分步运行指南”；`notebooks/main_pipeline.ipynb` 整合 W1-W4 全流程                                 |

### 14. AI 工具使用声明

声明：本项目在实现与写作过程中使用了 AI 工具，主要用于代码实现、文档整理与一致性复核；核心建模与分析由本人主导，AI 参与完成的内容均经本人逐项审查与核实。

#### 一、工具名称、版本与型号

OpenAI Codex 桌面版（应用版本 26.915.31945，命令行内核 codex-cli 0.155.0-alpha.9.2），本次会话调用的大语言模型为 deepseek-v4-flash（经本机服务转发）。除此之外未使用其他 AI 工具，也未用 AI 生成或补全任何数据。脚本文件头部均注明所用工具与版本。

#### 二、在哪个环节、为了什么用的

本人按“提要求、定口径、验结果”的方式使用它：每步都说清要算什么、口径边界在哪、结果长什么样算通过，AI 承担执行与整理工作。具体落在四类环节。

代码实现：每个脚本算什么、输入输出是什么、参数怎么取、结果怎么算通过，全部由本人定义；AI 编码工具负责把定义翻译成代码，含重构与绘图。W1 清洗与 EDA、W2 特征工程与基线、W3 调参与集成、W4 评估与期望损失框架的脚本均按此流程产出。

工程组织：在 AI 辅助下把 W1 至 W4 的脚本串成可复现流程、生成 Notebook 骨架、整理产物目录与命名；目录结构与交付边界由本人确定。

文字与图表：报告、周报与演示文稿的脚本由 AI 辅助总结，本人逐节编写、改写、逐项核对数据后定稿；图表绘制与版式脚本在 AI 辅助下完成。

一致性复核：借助 AI 交叉比对同一数字在报告、仪表板与演示文稿中的取值，列出待确认清单，再由本人核对裁定。

模型结构、超参数、金额口径与业务结论不是 AI 定的，见第四条。

### 三、主要提示方式与互动过程

提问按“先定口径、再给约束、后验输出”的顺序写：每一步先说清要算什么、口径与边界（如只用 train 建模、先切分再拟合以防泄漏），要求 AI 给出可运行的脚本与中间输出；跑完由本人检查结果是否符合预期，对不上就回去改口径或改实现。典型提问原文（节选）：

“只用 train 建模，testA 不参与；划分先切分再拟合，检查有没有泄漏。”

“把净息差、回收率做成可调参数，重算阈值敏感性，输出明细 CSV。”

“按期限拆两条线时，确认金额在期望利润公式两边是否约掉，给出推导过程。”

“核对报告、仪表板、演示文稿里同一个数字是否一致，不一致的标出来源。”

### 四、采纳了什么、改了什么、核验了什么

AI 产出的脚本由本人运行并复现，逐项核对输入、输出与关键参数；输出与预期不符时，先定位是口径错了还是实现错了，改到一致才进入交付。报告中每个数字回溯到 outputs/ 下的脚本输出与明细 CSV 后才写进正文。

代码层面由本人拍板的处理（节选）：数据划分与防泄漏顺序（先切分再拟合）、清洗的三类通道（不可能值 / 缺失 / 真实极端值）、类别编码与收入 log 变换、38 个特征的取舍、调参方法与搜索预算、阈值扫描的步长与范围、期望利润公式的写法与口径、复审带 20% 带宽的定义。

以下是被本人纠正过的实例（节选）：

概率刻度：校准轴与模型原始轴是两套刻度（原始轴 0.553 对应校准轴 0.2406），AI 初稿曾把两轴数值并排使用，本人推导核对后确认不能混用，报告与仪表板分别标注。

期望利润公式：AI 起初只说“按期限拆线”，本人从公式推出金额在两边同时出现、不改变判定，才确定按期限拆两条线这个结论。

时间外验证：AI 初稿写成“拿未来数据验证”，本人纠正为按放款日期切分的时间外样本，并说明它只用于稳健性检查。

图表与文案：AI 生成的 16 行敏感性表与部分说明可读性差，本人改成热力图并重写说明。

数字一致性：报告、仪表板与演示文稿曾出现 0.55 / 0.553、6,340 / 6,344 万元等口径混用，由本人逐条回溯 CSV 后统一。

结论归属：研究设计、数据划分与防泄漏规则、模型与超参数选择、金额口径与业务结论（净息差 6%、回收率 30%、起手线 0.55、按期限拆两条线、复审带 20% 带宽）均由本人决定并写明理由；文字由本人编写确认后定稿；文献由本人提出需要覆盖的方法方向，AI 辅助检索得到候选清单，本人逐项阅读文献内容，确认可用后再逐条核对 DOI、arXiv 编号、卷期页与链接可访问性，剔除对不上或打不开的条目后定稿（见第 15 节）。

未使用 AI 的边界：不用于生成或补全数据，不用于编造文献与结论，不用于替代业务判断；不使用外部敏感数据，也未向任何外部服务提交信息。

责任声明：本报告的内容、结论与可能存在的错误由本人负责。

## 15. 参考文献

[1] 阿里天池. 贷款违约预测数据集[DB/OL]. <https://tianchi.aliyun.com/dataset/140861>.

- [2] KE G, MENG Q, FINLEY T, et al. LightGBM: a highly efficient gradient boosting decision tree[C]//Advances in Neural Information Processing Systems 30. 2017. [https://proceedings.neurips.cc/paper\\_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html](https://proceedings.neurips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html).
- [3] CHEN T, GUESTRIN C. XGBoost: a scalable tree boosting system[C]//Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. 2016: 785-794. DOI: [10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785).
- [4] BREIMAN L. Random forests[J]. Machine Learning, 2001, 45(1): 5-32. DOI: [10.1023/A:1010933404324](https://doi.org/10.1023/A:1010933404324).
- [5] LUNDBERG S M, LEE S I. A unified approach to interpreting model predictions[C]//Advances in Neural Information Processing Systems 30. 2017. [arXiv: 1705.07874](https://arxiv.org/abs/1705.07874).
- [6] PLATT J. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods[M]//Advances in Large Margin Classifiers. Cambridge: MIT Press, 1999. <https://www.cs.colorado.edu/~mozer/Teaching/syllabi/6622/papers/Platt1999.pdf>.
- [7] ZADROZNY B, ELKAN C. Transforming classifier scores into accurate multiclass probability estimates[C]//Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. 2002: 694-699. DOI: [10.1145/775047.775151](https://doi.org/10.1145/775047.775151).
- [8] NICULESCU-MIZIL A, CARUANA R. Predicting good probabilities with supervised learning[C]//Proceedings of the 22nd International Conference on Machine Learning. 2005: 625-632. DOI: [10.1145/1102351.1102430](https://doi.org/10.1145/1102351.1102430).
- [9] GUO C, PLEISS G, SUN Y, et al. On calibration of modern neural networks[C]//Proceedings of the 34th International Conference on Machine Learning. 2017. [arXiv: 1706.04599](https://arxiv.org/abs/1706.04599).
- [10] BRIER G W. Verification of forecasts expressed in terms of probability[J]. Monthly Weather Review, 1950, 78(1): 1-3. DOI: [10.1175/1520-0493\(1950\)078<0001:VOFEIT>2.0.CO;2](https://doi.org/10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2).
- [11] PEDREGOSA F, VAROQUAUX G, GRAMFORT A, et al. Scikit-learn: machine learning in Python[J]. Journal of Machine Learning Research, 2011, 12: 2825-2830. <https://jmlr.org/papers/v12/pedregosa11a.html>.
- [12] HAND D J, HENLEY W E. Statistical classification methods in consumer credit scoring: a review[J]. Journal of the Royal Statistical Society: Series A, 1997, 160(3): 523-541. DOI: [10.1111/j.1467-985X.1997.00078.x](https://doi.org/10.1111/j.1467-985X.1997.00078.x).
- [13] LESSMANN S, BAESENS B, SEOW H V, et al. Benchmarking state-of-the-art classification algorithms for credit scoring: an update of research[J]. European Journal of Operational Research, 2015, 247(1): 124-136. DOI: [10.1016/j.ejor.2015.05.030](https://doi.org/10.1016/j.ejor.2015.05.030).

正文引用位置对照：

| 引用位置                 | 文献               |
|----------------------|------------------|
| 1.1 业务背景（信贷评分方法）     | [12][13]         |
| 2.1 数据来源             | [1]              |
| 6.2 三个基线模型           | [3][4][11]       |
| 7.5 最终模型选择（LightGBM） | [2]              |
| 8 模型解释（SHAP）         | [5]              |
| 9.5 概率校准             | [6][7][8][9][10] |

附录：关键数字速查

样本与划分：80 万 → 清洗后 798,498（47 列 → 38 特征）；训练 558,948 / 验证 159,700 / 测试 79,850，各集违约率约 20%。

最终模型：LightGBM（63 叶 / 200 棵 / 深度 8 / lr 0.05 / 子采样 0.8 / 列采样 0.7）；测试集 AUC 0.7249、KS 0.3269（阈值 0.5 下 precision 0.3254 / recall 0.6749 / F1 0.4390）；三集 AUC 0.7432 / 0.7294 / 0.7249；时间外 AUC 0.7184、KS 0.3190。校准后平均概率 0.1993（实际 0.1995）、Brier 0.2106 → 0.1424，AUC 不变。

金额（中性口径，净息差 6% / 回收率 30%）：全放 2,085 万；单一线 0.50 / 0.55 / 0.553 → 6,155 / 6,340 / 6,344 万（验证集 0.560 → 6,342 万）；拆线（3 年期 0.50、5 年期 0.62）→ 6,611 万，多 267 万元。

定价（中性口径）：概率 <5% 的 7,594 人让价空间 5.6 / 5.1 个百分点，利率下浮 1 个百分点让价 316 万元；20% 以上两档无空间。